Artificial Intelligence and Machine Learning

Project Title: A Comprehensive Measure of Well-Being (HDI Prediction & Analytics System – HDI Intelligence)

Revision: 28 October 2026

Team Members:

1. **Madireddy Meena Mahanth Harinadh** (Machine Learning Engineer)

Dataset preparation and preprocessing, exploratory data analysis, country-separated comparison of regression models (Linear Regression, Decision Tree, Random Forest and Extra Trees), export of the selected model for reproducible inference, and documentation of model results, evidence and limitations.

Academic context: AI & ML internship through SmartBridge Educational Services Pvt. Ltd. in collaboration with APSCHE (120 hours over two months).

Introduction:

The Human Development Index (HDI) is an internationally recognized measure developed by the United Nations Development Programme (UNDP) to assess a country's overall level of development. It evaluates three key dimensions of human well-being: health, education, and standard of living. This project presents published human-development observations alongside a reproducible machine-learning reconstruction of the HDI. The current application is a cyan React and TypeScript website, branded HDI Intelligence, built with Python, scikit-learn, Pandas and NumPy for the offline research workflow, and React 19, Vite and Recharts for the public interface.

Project Description:

The HDI Prediction & Analytics System is an AI-assisted web application that estimates the Human Development Index of a country from four socio-economic indicators using Machine Learning. Economic output alone cannot describe the opportunities people have to live long, educated and materially secure lives. HDI therefore combines life expectancy, education and Gross National Income (GNI) per capita into one bounded index. A score closer to one indicates higher achievement in those dimensions, but it does not capture every inequality, environmental cost or personal experience.

The practical problem addressed is making a published multi-dimensional development measure easy to explore and interpret, without disguising an index-reconstruction experiment as independent forecasting. Static spreadsheets make comparison cumbersome, while an attractive interface can still mislead if its data mappings, charts or claims are wrong. This project therefore keeps observations, model estimates and hypothetical scenarios visibly separate.

The four active inputs

- **Life expectancy at birth** – represents health.
- **Expected years of schooling** and **mean years of schooling** – represent education for different cohorts.
- **GNI per capita** (constant 2021 PPP dollars) – represents standard of living.

Population, public expenditure and region are not numerical features of the current model; country and region labels support exploration only. Based on the estimated score, the application classifies a country into one of four development categories:

HDI category	Threshold used in the application
Low Human Development	Below 0.550
Medium Human Development	0.550 to below 0.700
High Human Development	0.700 to below 0.800
Very High Human Development	0.800 and above

Main capabilities

- **Explore Data** – published UNDP HDR 2025 country records, two-country historical trends (1990–2023), searchable indicator table and JSON export.
- **HDI Predictor** – validated four-indicator input, local ML estimate, direct formula reference, dimension bars, sensitivity view and JSON export.
- **Scenarios** – baseline versus hypothetical indicators using the same estimator, with reset and non-causal labelling.
- **Methodology** – data source, model selection rule, held-out metrics, predicted-versus-published scatter plot and limitations.
- **Optional AI explanations** – prose explanations through Groq using the visitor's own key, held in tab memory only. The numerical estimate never comes from the language model.

Scientific boundary

Reconstructing current HDI from contemporaneous components is not forecasting future HDI. Hypothetical changes to input indicators do not prove the effect of government spending or any other intervention. AI explanations are optional prose, not the numerical estimator.

Original approach versus current implementation

Concern	Original (first approach)	Current application
Frontend	Flask/Jinja and Bootstrap	React, TypeScript, Vite and cyan CSS
Inference	Python model on the server	Exported estimator in shared JavaScript
Data	Earlier experimental panel	Pinned official UNDP HDR 2025 snapshot
Identity / storage	Accounts and database	No account or database; transient state
AI	Recommendation narrative	Optional Groq explanations with personal keys
Deployment	Render / WSGI	Vercel static frontend plus Node function

System Architecture and Data Entities:

The application has two execution boundaries: a static browser application and an optional same-origin Node function. Offline Python training produced the committed artifacts; it is not part of ordinary website requests. The browser loads countries.json, model.json and evaluation.json, and lazily loads explorer.json for the historical explorer. Shared inference code validates the four inputs, computes the trained estimate and calculates the dimension indices. The optional API imports the same model artifact and inference module to reconstruct the numerical evidence before requesting prose from Groq.

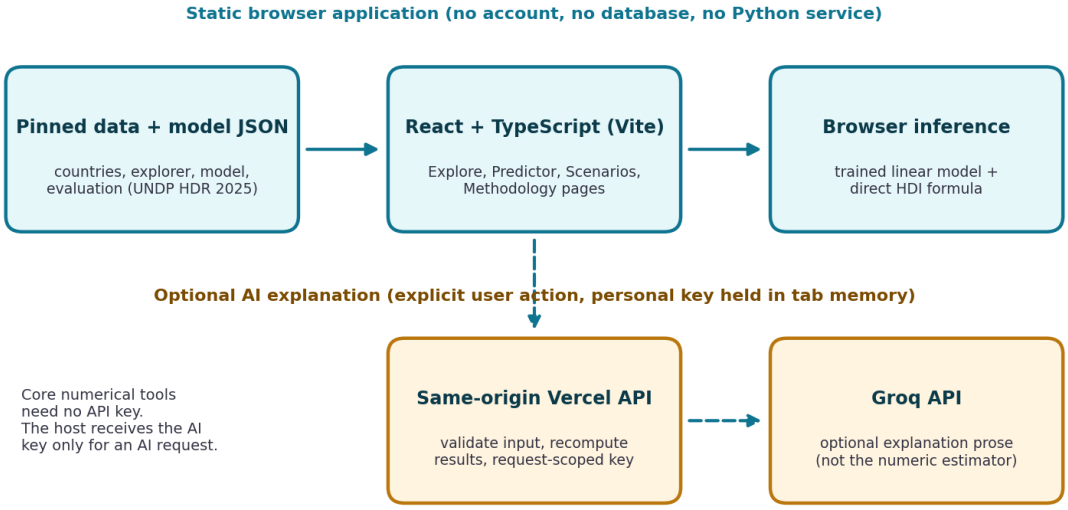


Figure 1. Current React, inference and optional Groq architecture.

Because the earlier Flask/database design has been replaced, there are no active User, Recommendation or Report database tables. The current data model consists of JSON artifacts and transient state:

Entity	Fields or contents	Lifetime
Country	ISO3, name, source region, 2023 indicators, published HDI and history	Committed public data
Country-year	ISO3, year, four indicators and observed HDI	Committed complete-case panel
Model	Feature order, coefficients, intercept, medians, observed ranges	Committed trained artifact
Evaluation	Split groups, comparisons, held-out metrics, samples, limitations	Committed experiment evidence
Draft / baseline / scenario	Four validated inputs; baseline changes only on calculation	React memory
Personal AI credential	Masked entry, current status, request revision	Active tab memory only
Export	Selected records or prediction inputs and outputs	User-requested JSON download

Consistency rules

- Country selection loads a published preset. Editing fields does not silently change the committed result; a stale-result notice requests another calculation.
- A newly calculated baseline resets the scenario. Historical presets carry their source year into the predictor.
- Removing or replacing an AI key cancels pending browser work and clears the old explanation. Refreshing the page clears the key and transient input state.
- Only the light/dark theme preference uses localStorage, separately from any credential.

Pre-requisites:

Before developing the system, the required hardware, software, programming knowledge, development tools and machine learning environment must be established. Ordinary use requires only a modern browser and the committed static data. Local development uses Node 24.x and the repository lockfile; Python is needed for the preserved offline research workflow, not for entering indicators or viewing results.

1. Hardware Requirements

- Processor: Intel Core i3 (8th generation) / AMD Ryzen 3 or above.
- RAM: 4 GB minimum, 8 GB or more recommended.
- Storage: about 10 GB free; SSD recommended.
- No GPU is required for the exported linear estimator. These are practical recommendations, not measured benchmarks of the application.
- Stable internet connection for installing packages and downloading the dataset.

2. Operating System

Platform-independent: Windows 10/11, Ubuntu Linux or macOS.

3. Programming Languages

- **Python** for dataset preparation, training and evaluation.
- **TypeScript / JavaScript** for the React interface, shared inference module and optional Node API.

4. Development Environment

- Python 3.x, Jupyter Notebook or Anaconda (optional) for experiments.
- Node 24.x with npm and the committed lockfile.
- Visual Studio Code with Python and TypeScript support.

5. Libraries and Technology Stack

Layer	Implementation	Purpose
UI	React 19 and TypeScript	Accessible, state-driven pages
Build	Vite 7; Tailwind CSS 4 integration	Development server and static output
Visualization	Recharts 3	Historical areas, scenario bars, sensitivity and scatter plot
Styling	Cyan tokens and responsive CSS	Shared light/dark identity, reduced motion
Inference	shared/inference.mjs	Validation, model, formula, categories, sensitivity
Optional API	Node function on Vercel	Origin validation and Groq requests
Research	Python, Pandas, NumPy, scikit-learn	Dataset preparation and regression experiment
Verification	TypeScript, ESLint, Node tests, Playwright, axe	Static, numerical, API and browser checks

6. Dataset Requirements

The dataset must contain reliable indicators – life expectancy, expected years of schooling, mean years of schooling, GNI per capita and published HDI – identified consistently by ISO3

code, and should be complete, consistent, accurate and free from duplicate country-year records.

7. Machine Learning Concepts

Data collection and cleaning, complete-case selection, logarithmic feature transformation, country-grouped train/validation/test splitting, regression algorithms, model evaluation (MAE, RMSE, R^2), permutation importance and model serialization.

8. Version Control and Deployment Tools

Git and GitHub for source control (the delivered workspace is a downloaded archive without .git, so no repository history is claimed). Vercel is used for the static frontend and optional Node function.

9. Soft Skills and Basic Knowledge

Problem solving, analytical thinking, documentation and debugging skills; basic knowledge of Python, typed component state, HTTP boundaries, accessible HTML, descriptive statistics and held-out evaluation.

PROJECT FLOW:

Development follows a structured workflow: environment setup, dataset collection and preprocessing, exploratory analysis, country-separated splitting, model development and evaluation, reproducible model export, web application development, state and privacy design, verification and deployment. Each epic below keeps the original academic progression, updated to the current implementation.

Epic 1: Environment Setup and Package Installation

Objective

Prepare a reproducible environment. Work from the active application folder, install locked dependencies and start the local frontend/API adapter.

Activities

- Install Node 24.x and Python 3.x
- Install Visual Studio Code
- Install locked dependencies with npm ci
- Start the development server

```
cd "8. Project Demonstration"
npm ci
npm run dev
```

Open http://127.0.0.1:5173. The local Node adapter listens on 127.0.0.1:3001. On Windows, if PowerShell blocks npm.ps1, use npm.cmd ci and npm.cmd run dev without changing machine-wide execution policy.

Why is this Epic Important?

A configured environment ensures every run uses the same software versions, so the application can be executed and tested reliably.

Expected Output

- Development environment successfully configured
- All dependencies installed
- Local website and optional API route running

Epic 2: Libraries and Application Initialization

Objective

Import the libraries and initialize the application. The React entry point mounts App inside StrictMode. App loads the three initial JSON assets (countries, model, evaluation) and provides retry feedback if loading fails. The Explorer lazily loads its larger historical panel only when opened.

Activities

- Mount the React application
- Load countries.json, model.json and evaluation.json
- Import numeric inference from the shared module
- Configure ignored .env values for the local API (never prefixed with VITE_)

Expected Output

- Application initialized and static artifacts loaded
- No browser pickle loading or server Python process required

Epic 3: Dataset Collection and Understanding

Objective

Collect a reliable HDI dataset and understand its structure before preprocessing. The research snapshot is the official UNDP Human Development Report 2025 complete time series; its source URL and SHA-256 are recorded in evaluation.json, so the experiment stays reproducible even if the external source is later revised.

Artifact fact	Recorded value
Source release	UNDP Human Development Report 2025
Complete country-year records	5,940
Countries	193
Observation years	1990–2023
Excluded incomplete rows	622
Raw source SHA-256	61ed82e5b66c88dfca8ff9fac775c63981ecab6a254862af97acacc41c143117

Dataset Features

Target: published HDI. Inputs: life expectancy, expected schooling, mean schooling and GNI per capita. ISO3 identifies countries; names are display labels; regions use the source grouping (including “Other / not assigned”).

Coverage limitations

A complete-case dataset is not a claim that every country has every year. Missing observations remain gaps, and records missing any of the four indicators cannot supply a predictor preset. Country-year records are not statistically independent.

Expected Output

- Dataset loaded and features/target identified
- Source, coverage and limitations documented

Epic 4: Data Preprocessing

Objective

Transform the raw time series into a clean country-year panel suitable for machine learning. The training script reshapes source columns, excludes regional aggregates, drops rows missing any model input or HDI, and checks duplicate country-year keys and the valid HDI interval. The 622 excluded rows come from this existing experiment.

Numerical transformation

GNI is transformed with the natural logarithm; the other three inputs keep their original units. The feature order is fixed in model.json. This selected four-feature model uses no StandardScaler, median imputation or categorical encoding.

```
x = [life_expectancy, expected_schooling, mean_schooling,
ln(gni_per_capita)]
y_hat = clip(intercept + sum(coefficients[i] * x[i]), 0, 1)
```

Validation and extrapolation

The interface rejects empty, non-finite and out-of-bound inputs: life expectancy 20–95, expected schooling 0–25, mean schooling 0–20 and GNI 100–200,000. These bounds are not claims that training data covers every combination; the model's observed ranges separately trigger an extrapolation warning. Country presets are rounded to two decimals for entry.

Why is this Epic Important?

Clean, consistent data prevents bias and training errors, and a fixed feature order guarantees training and inference agree.

Expected Output

- Clean complete-case panel
- No duplicate country-year keys
- Model input transformation defined

Epic 5: Exploratory Data Analysis (EDA)

Objective

Examine the published data without concealing missingness. The website provides historical HDI trends, two-country comparisons, dimension summaries and a searchable indicator table.

Activities

- Dataset overview and statistical summary
- Historical trend plots for any two countries
- Dimension (health, education, income) summaries
- Searchable, filterable indicator table

The temporal chart includes every calendar year in the selected range. Values must be finite and within [0, 1]; absent observations are null and series never connect across missing years. A one-year selection shows visible point markers, and an exact-value table is the accessible alternative to the chart.

Expected Output

- Statistical insights and trend charts
- Visible empty and missing-data states

Epic 6: Country-Separated Train-Test Split

Objective

Evaluate on unseen countries. The experiment uses GroupShuffleSplit by ISO3: an outer split (seed 42) reserves test countries, and an inner split (seed 43) separates training and validation countries, so one country's repeated annual rows never appear in both training and a held-out group.

Partition	Rows	Purpose
Training	3,611	Fit comparison models
Validation	1,178	Select lowest MAE

Partition	Rows	Purpose
Test	1,151	Evaluate the selected, refitted model once

The winner is refitted on training plus validation countries before final evaluation. Grouping reduces repeated-country leakage but does not remove dependence between economies or years. A separate 2019–2023 temporal diagnostic uses contemporaneous indicators and is not a forecast.

Epic 7: Machine Learning Model Development

Objective

Train regression models and select the best algorithm by validation MAE. Four families were compared on countries excluded from training.

Model	Validation MAE	Validation RMSE	Validation R^2
Linear regression (selected)	0.004148	0.005374	0.998855
Extra Trees	0.004209	0.005886	0.998626
Random forest	0.007959	0.011751	0.994523
Decision tree	0.015369	0.020814	0.982815

Linear regression has the lowest recorded validation MAE and a compact coefficient-based form, using a logarithmic income feature and bounded outputs. Choosing a more complex algorithm merely because it appears more advanced would not follow the recorded selection rule. All comparisons are read from the committed evaluation artifact; no model was retrained or tuned for this documentation.

Expected Output

- Trained and compared regression models
- Linear regression selected by the recorded rule

Epic 8: Model Evaluation and Comparison

Objective

Evaluate with MAE (average absolute error in HDI points), RMSE (weights large errors more) and R^2 (explained variation, not a percentage accuracy). The model and the direct formula are evaluated against published HDI on the same held-out rows.

Evaluation	MAE	RMSE	R^2
Held-out countries (model)	0.004404	0.006670	0.998183
Direct formula, same rows	0.000247	0.000286	0.999997
Temporal diagnostic	0.004257	0.006712	0.998006

Strong reconstruction scores are expected because HDI is defined by the supplied components. The formula’s lower error is not a failure; it is the essential baseline that prevents overstating the learned approximation. These scores do not validate forecasts, causal recommendations, calibrated uncertainty intervals or performance on every out-of-range input.

Formula reference and local sensitivity

```
Health    = cap((life_expectancy - 20) / 65)
Education = (cap(expected_schooling / 18) + cap(mean_schooling / 15)) / 2
Income    = cap(ln(gni_per_capita / 100) / ln(750))
HDI_formula = cube_root(Health * Education * Income)
```

The predictor also replaces one input at a time with its training median and shows the change in the estimate. These effects are neither SHAP values nor additive contributions. Permutation importance (eight repeats) reports the increase in test MAE after shuffling a feature; correlated inputs limit its interpretation.

Expected Output

- Comparison tables of models, formula and held-out metrics

Epic 9: Model Saving and Reproducible Inference

Objective

Save the selected model for deployment. It is exported as JSON (schema version 1) with feature order, fitted coefficients, intercept, medians and observed ranges. The same JavaScript module is imported by the browser and the API. Existing reference data holds 1,407 Python prediction cases, so JavaScript parity is checked without retraining. The original pickle artifacts are preserved in the historical backup and are never served or deserialized during visitor interaction.

Expected Output

- model.json with schema version 1
- Parity file parity.json (1,407 cases)

Epic 10: Web Application Development

Objective

Build the public interface that lets users interact with the model. The original Flask stage is retained as a project-flow step, but the implementation is now React. App owns the committed baseline and scenario state; the navbar exposes five public routes; shared controls render validated inputs, categories, score scales and dimension bars.

Component	Responsibility
Trend	Published history, year range, gaps, tooltips and exact-value table
Predictor	Draft inputs, committed estimate, formula and export
Scenarios	Baseline/update comparison and reset
AIContext / AISettings	Transient personal credential and status
Insights / Explanation	Explicit request and safe prose/table rendering

Expected Output

- Working public HDI website with Home, Explore Data, HDI Predictor, Scenarios and Methodology pages

Epic 11: State and Privacy (replaces Database Integration)

Objective

The original database epic is intentionally revised: there is no active user database or stored prediction history. Selections and inputs live in React state, data and model artifacts are static, and downloads occur only on request.

Bring Your Own Key

AI Settings provides masked entry, show/hide, use, change and remove controls. Saving marks a key as configured, not verified; only a successful explanation request changes status to connected. Removing or changing it aborts pending browser responses and clears the old explanation.

Server protections

- POST /api/insights validates method, content type, exact origin, key format, four numeric inputs and a question of at most 500 characters; body limited to 4 KiB.
- Instance-local limit of six requests per minute per IP; numerical evidence is recomputed; 15-second provider timeout; bounded response text.
- Distinct safe errors for rejected credentials, rate limits, timeouts and provider outages; provider error bodies are never forwarded; no fallback to an owner account after a personal-key failure.
- Owner-key access is disabled by default and needs ALLOW_OWNER_AI=true, a server-only GROQ_API_KEY and a strong AI_ACCESS_TOKEN. The default model is openai/gpt-oss-20b (configurable with GROQ_MODEL).

Expected Output

- No stored credentials or user records; transient, request-scoped key handling

Epic 12: Testing and Validation

Objective

Verify that every module functions correctly. Phase 4 verification addresses the reported chart failure and the personal-key contract.

Check	Recorded Phase 4 result
Numerical and API tests	21 passed; 1,407 parity cases, maximum error 2.22e-16
TypeScript	Passed; production build also runs tsc -b
ESLint	Passed
Production build	Passed; Vite 7.3.7, 2,182 modules
Chrome functional checks	11 distinct tests passed
Production Chrome check	1 passed; built assets, route refresh, offline inference after load
Actual screenshots	10 captures with real local data, no credentials, no provider request

Browser coverage includes Cyprus/China, India/United States and Norway/Afghanistan across several years, single-year points, South Sudan missing-data behaviour, nonzero SVG drawing widths, desktop/mobile and light/dark plots, table filters and exact exports, predictor calculation, scenario deltas and reset, and menu, theme and modal interactions.

AI verification boundary

API unit tests use injected provider responses, and browser AI tests intercept the route with labelled test fixtures. These are not real Groq explanations. No live provider success and no Vercel deployment is claimed. Passing these checks does not prove behaviour in every browser, operating system or hosting environment.

Expected Output

- All recorded checks passing

Epic 13: Deployment

Objective

Prepare the application for public access. Vercel configuration is preserved for the static frontend and optional Node function; deploy from the application folder, not the repository root. Deployment itself remains the project owner's responsibility and has not been performed or verified.

Setting	Value
Root Directory	8. Project Demonstration
Install / Build command	npm ci / npm run build
Output Directory	dist
Function	api/insights.mjs
Production APP_ORIGIN	Exact HTTPS site origin, without trailing slash
Owner fallback	Disabled unless explicitly opted in and token-protected

The public frontend embeds no provider secret. Each visitor supplies a personal key for an explicit AI operation, and the host sees it while making the Groq request, so visitors should only use a site they trust. Browser code refuses to send a key from a non-HTTPS site except local loopback. Hash routes support refresh on a static host. The process-local throttle resets on cold starts and is not a durable global quota.

Expected Output

- Live Application: to be deployed and verified by the project owner
- Public URL: to be added after deployment

Key Features & Application Walkthrough

Landing Page :

Home introduces the index and its three dimensions with a custom three-node logo and top navigation. Published-country data, source-backed dataset totals (193 countries, 5,940 complete records, 1990–2023) and a historical trend replace administrative counters or invented statistics. Predict HDI loads the selected country into the predictor; Explore Global Data opens the explorer.

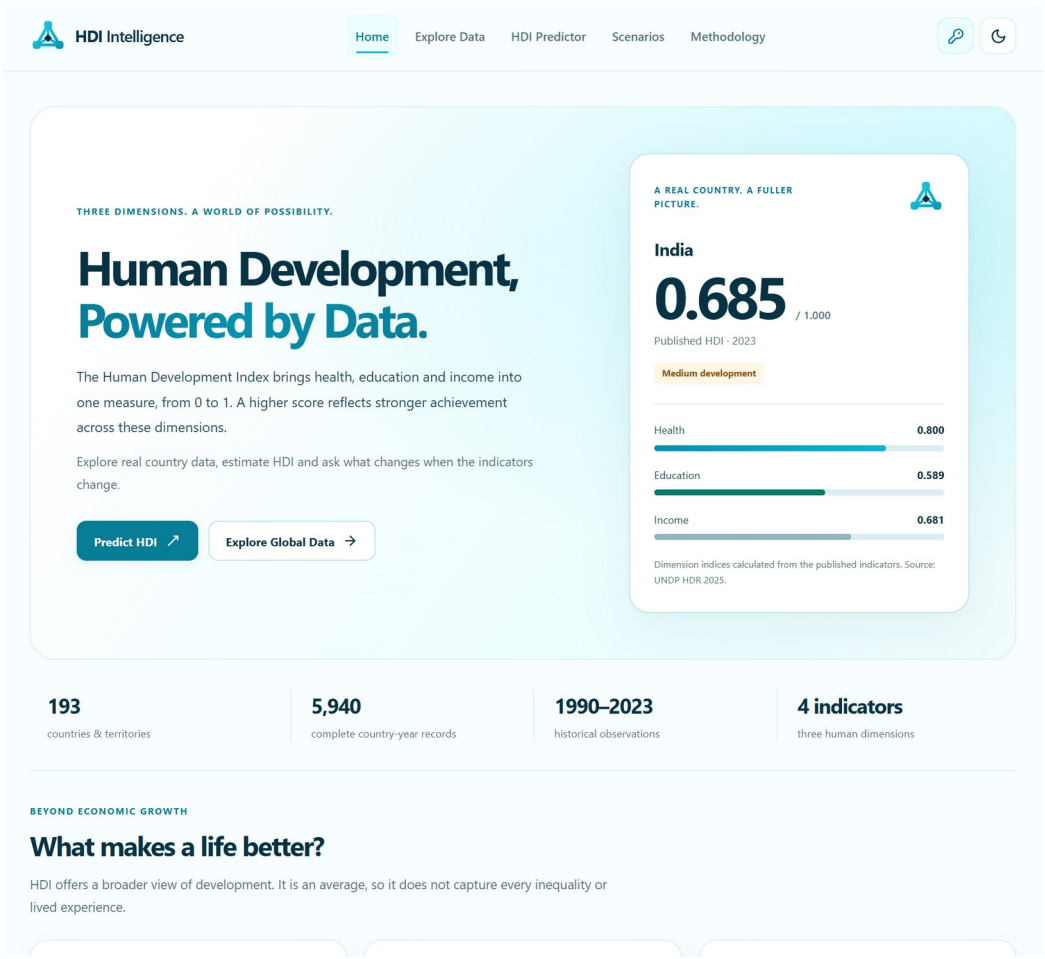


Figure 2. Current homepage with the HDI Intelligence logo, navigation and a real country summary.

Explore Data :

The explorer combines region, country, observation-year and comparison controls. Selecting a year limits the trend through that year and updates both country summaries and the indicator card. The table filters independently by year, region, search and development category; Export filtered data downloads exactly those records as JSON, and Use these indicators transfers a complete observation into the predictor with its source year.

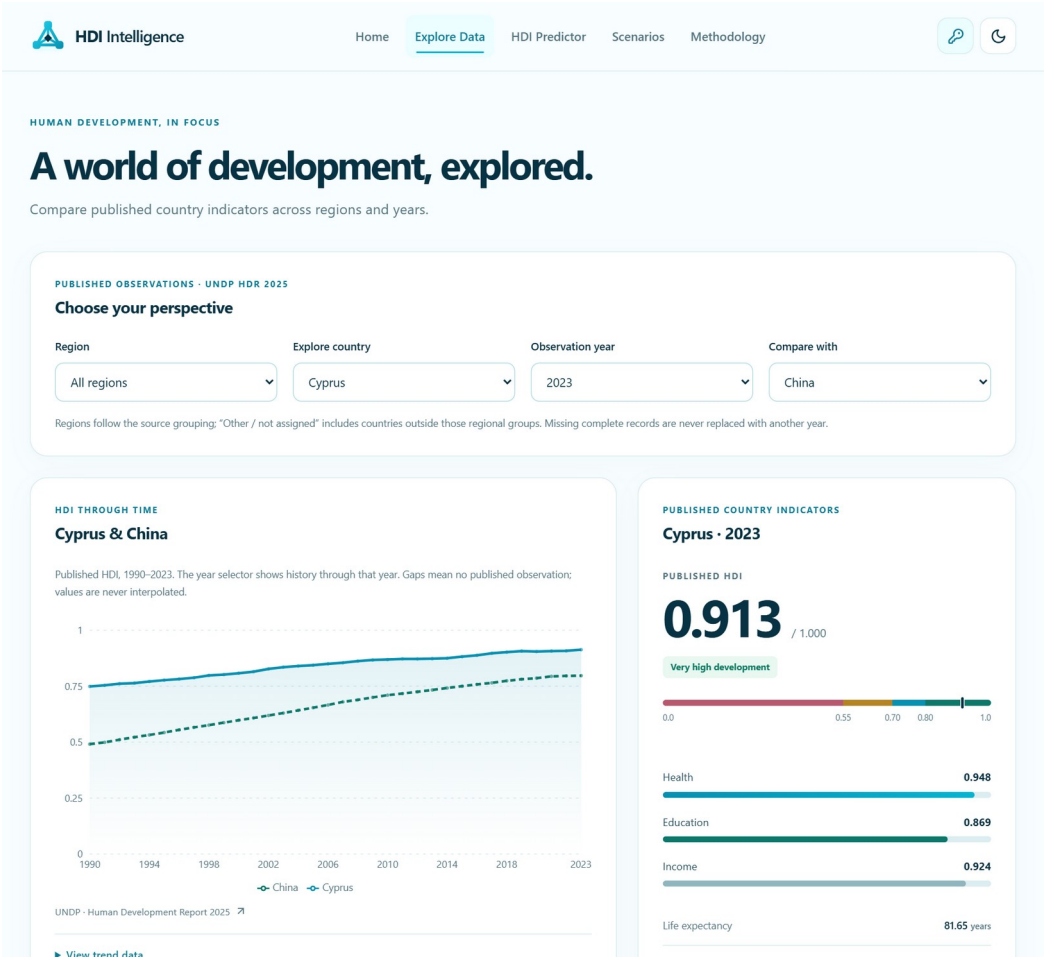


Figure 3. Explore Data with Cyprus and China selected.

Historical HDI Visualization Repair :

The reported Cyprus/China chart had valid data, series paths, legends and colours, yet the drawing SVG had zero width. A shared CSS rule applied max-width:100% to .recharts-wrapper, whose measurement parent was zero-width, overriding Recharts' measured pixel width. Removing the constraint restored the plot without manufacturing data.

Diagnostic	Before repair	After removing the constraint
Outer chart width	Approximately 691 px	Approximately 691 px
Drawing SVG width	0 px	691 px
Historical records	34 per country	Same unchanged records

A regression check now measures the SVG drawing viewport rather than merely checking that a path exists. The history component also includes missing calendar years explicitly, never connects across absent observations, shows point markers for short ranges and supplies a visible empty state.

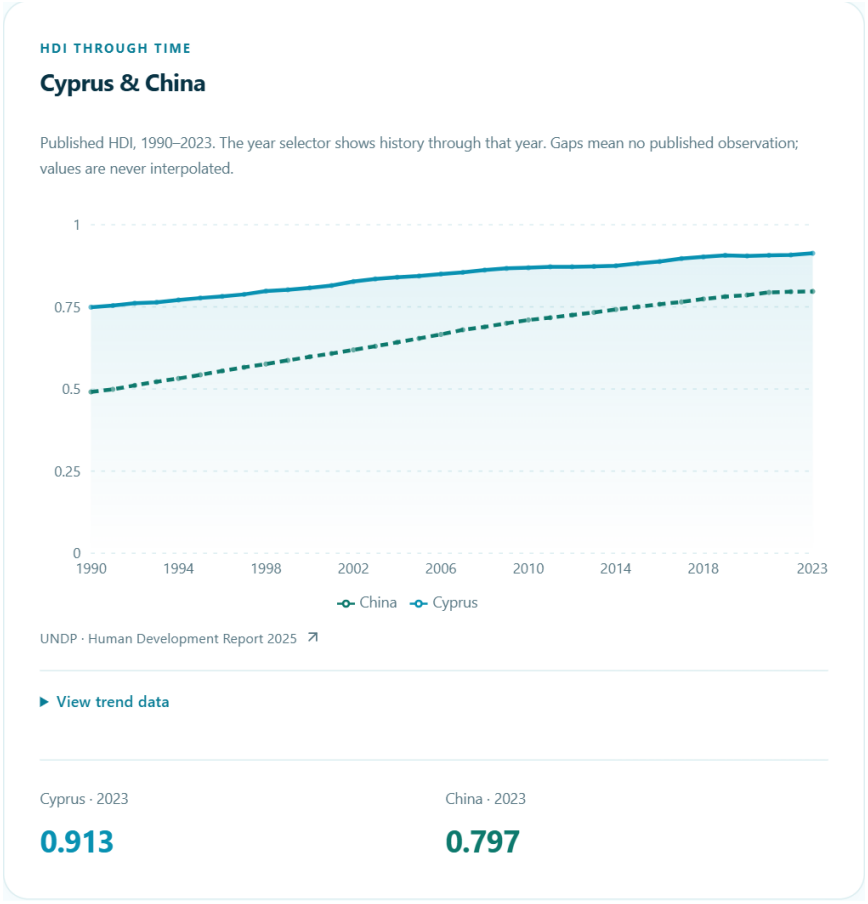


Figure 4. Published Cyprus/China trend after restoring the measured SVG width.

AI-Powered HDI Predictor :

The predictor groups health, schooling and income controls. Blank, non-numeric or out-of-bound entries prevent calculation, and a dirty-input notice shows when the displayed result still belongs to the previously committed baseline. The captured result is 0.680 / 1.000 (Medium development) against a formula reference of 0.684. Dimension bars show the health, education and income indices; Export result saves inputs, model estimate and formula reference. The estimate comes from the committed estimator, never from Groq.

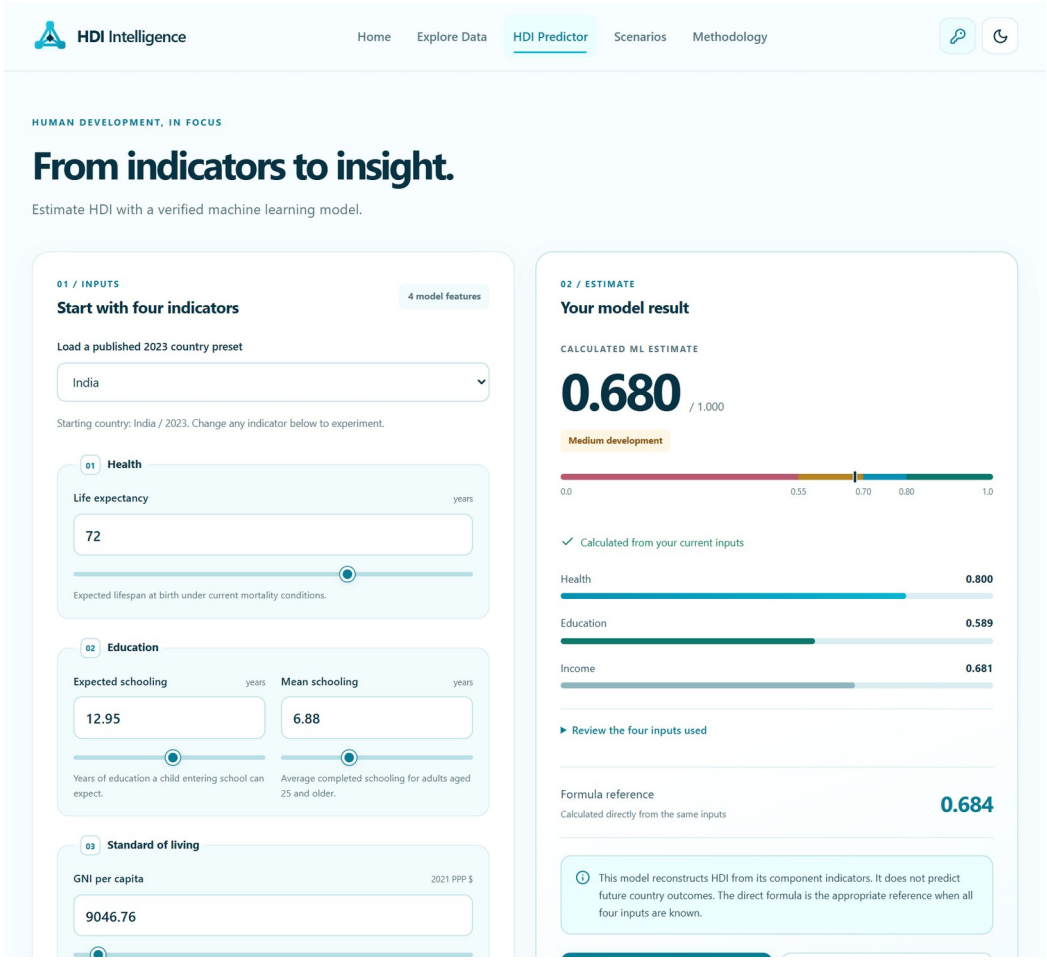


Figure 5. HDI Predictor using genuine model inference and the direct formula reference.

Scenarios :

Scenario analysis compares a committed baseline with modified indicators using the same estimator on both sides. Invalid inputs hide the chart and replace the delta with a correction prompt; Reset copies the baseline back. The captured comparison shows a change of +0.0692 HDI points. This is a hypothetical model comparison, not evidence that a policy would cause the change; the logarithmic income transformation and correlated inputs further limit interpretation.

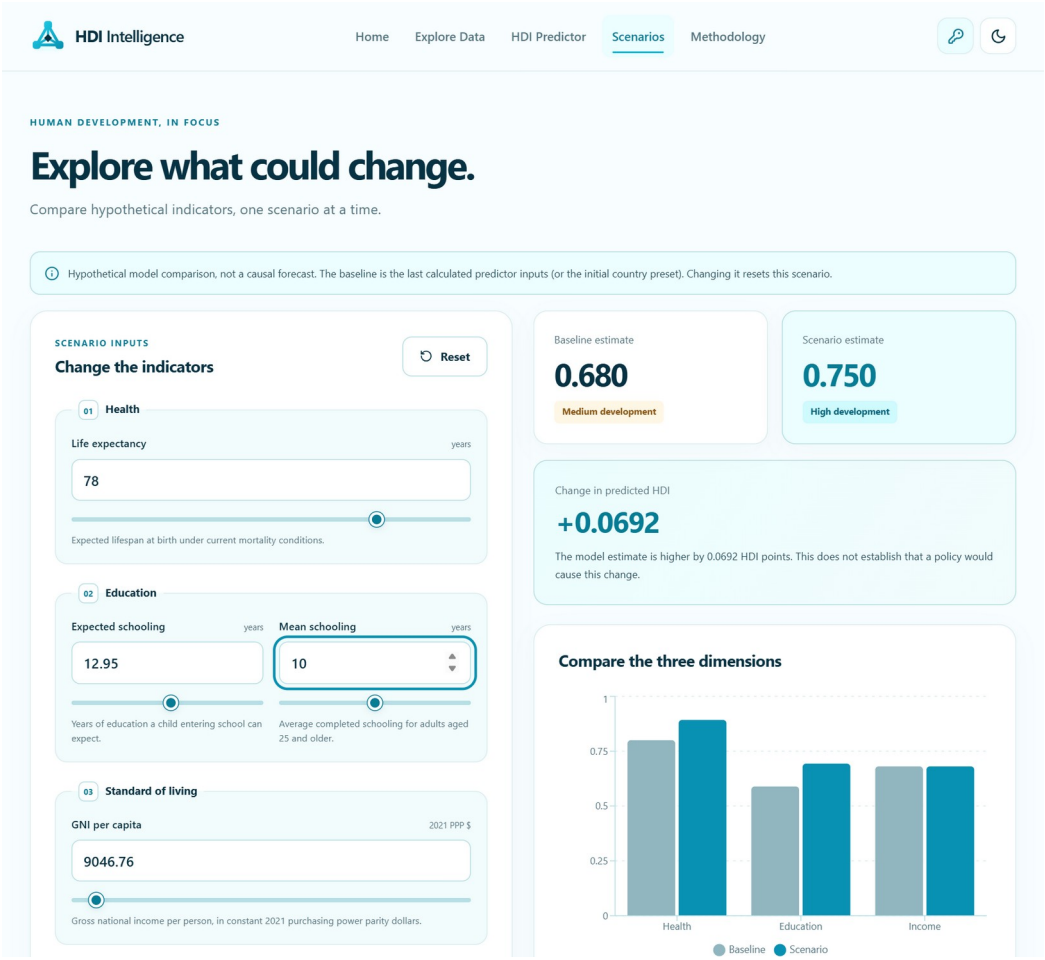


Figure 6. Scenario comparison with baseline, edited inputs and calculated differences.

Methodology :

Methodology makes the data source, selection rule and evaluation limits visible: held-out metrics (MAE 0.0044, RMSE 0.0067, R^2 0.9982), comparison-model validation errors, row partitions, a published-versus-predicted scatter plot and feature-reliance summaries. The diagonal is a visual reference, not an uncertainty band, and the scatter points are sampled from the committed held-out residual records.

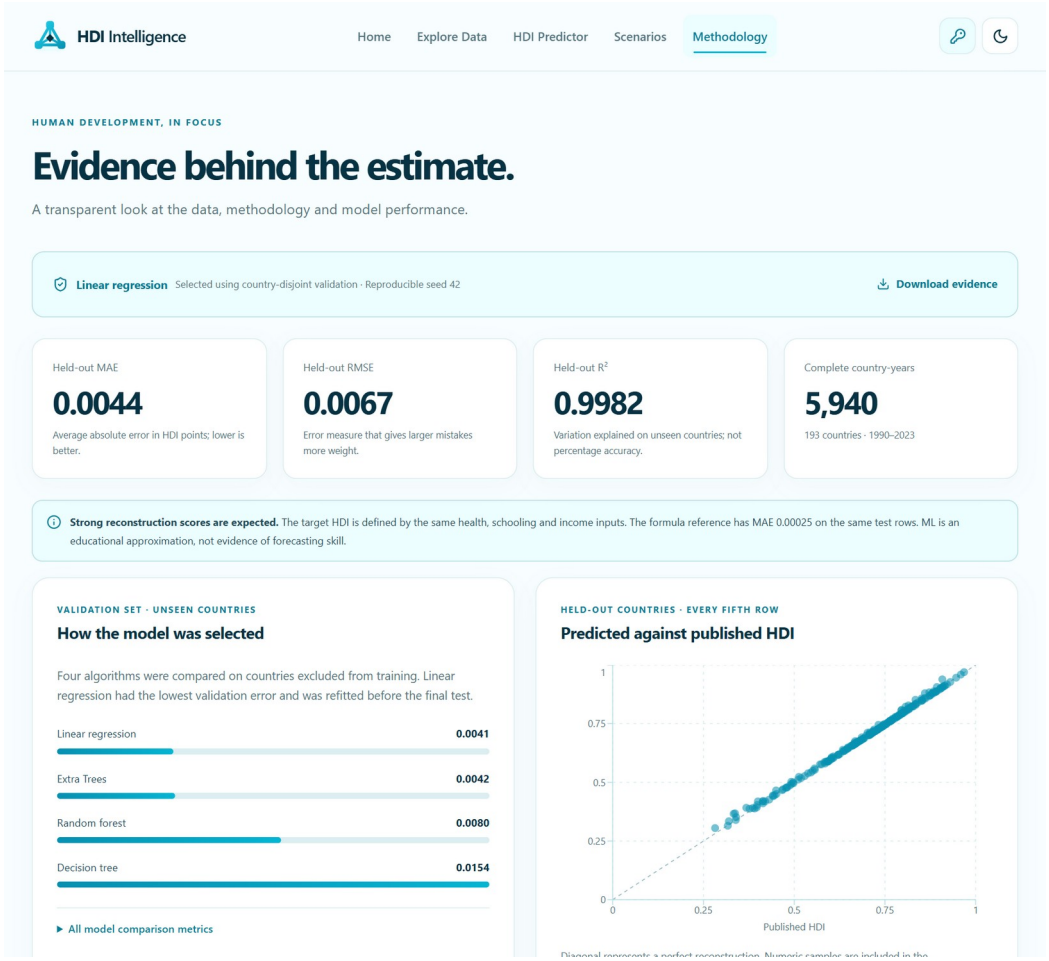


Figure 7. Methodology page with evaluation metrics and the held-out scatter plot.

Personal AI Settings :

The navbar key icon opens a dialog with keyboard focus containment and Escape dismissal. The key is masked by default and kept in memory for the current tab only; saving it does not contact Groq or imply it is verified. All data tools and predictions work without a key. No live AI-output screenshot is included, because mocked prose must not be presented as a real provider response.

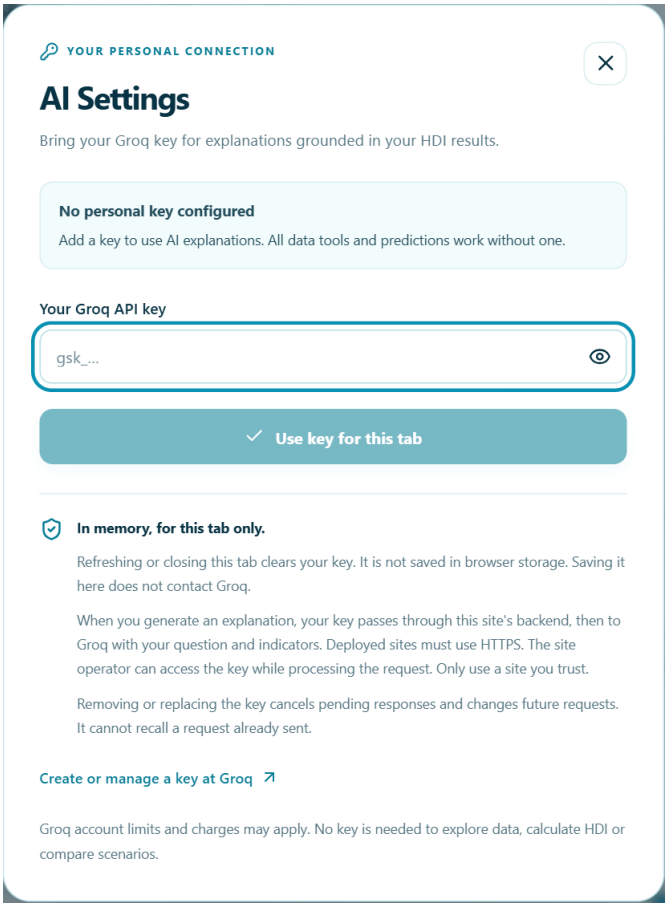


Figure 8. AI Settings with no credential entered or exposed.

Responsive Behaviour and Accessibility :

The cyan identity is retained on small screens and in dark mode. Navigation becomes a dismissible dropdown, wide tables scroll inside their cards and plots receive measured non-zero dimensions. Reduced-motion users get static results, and exact-value tables provide alternatives to visual trends. Checks covered 390-pixel mobile and desktop viewports in both themes; this is scoped evidence, not a claim of universal accessibility certification.

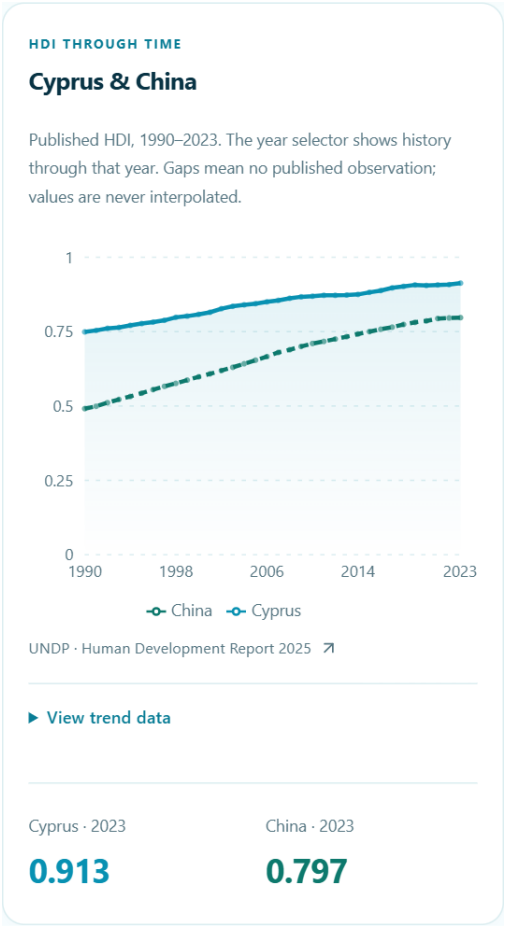


Figure 9. Actual mobile historical comparison with responsive chart sizing.

Limitations and Future Scope:

- HDI compresses multiple dimensions into an average and omits inequality and many environmental or institutional conditions.
- The estimator reconstructs a formula-defined index; complete-case filtering, source revisions, country dependence and extrapolation constrain interpretation. There are no calibrated confidence intervals, and local sensitivity is not causal attribution.
- AI text can be incorrect, depends on account access and may be rate limited; the site operator receives the key during a requested AI call.
- Live Groq success and actual Vercel deployment are not verified; browser checks cover the recorded engine and viewports only.
- Future work: a true forecasting study with lagged information and forward validation, durable public throttling, wider assistive-technology evaluation, data-release version selection and additional export formats.

Conclusion :

The HDI Prediction & Analytics System connects a reproducible regression experiment with an understandable public interface. Using the pinned UNDP HDR 2025 data (5,940 complete country-year records across 193 countries), country-separated validation selected linear regression, which reached a held-out MAE of 0.004404, RMSE of 0.006670 and R^2 of 0.998183, while the direct formula remains the better reference when all components are known.

The project demonstrates the complete machine learning lifecycle – data collection, preprocessing, exploratory analysis, model comparison, reproducible export, an interactive web application and verification – while keeping published observations, model estimates and hypothetical scenarios clearly distinct. The chart defect shows why valid data and DOM paths are insufficient evidence of a visible visualization; restoring measured SVG dimensions, preserving missing observations and testing real interactions aligned the implementation with its claims. The recorded checks support the implemented workflows, while the scientific and deployment limitations remain stated explicitly.

References

- United Nations Development Programme. Human Development Report 2025, documentation and downloads. https://hdr.undp.org/data-center/documentation-and-downloads
- UNDP. HDR25 Composite indices complete time series (pinned source file used by the experiment).
- Project artifacts: evaluation.json, model.json, countries.json, explorer.json, parity.json; code: shared/inference.mjs and ml/train.py.
- Groq Console: https://console.groq.com/

Thank You!

Thank you for reviewing this project.
